

Verilog Implementation of a Cache-Coherent, RISC-V Multicore Processor

Roshan Sundaram
 roshan.sundaram@duke.edu
Duke University
 Durham, NC
 April 2026



Abstract—As physical constraints limit the frequency scaling of single-core processors, modern compute performance relies heavily on multicore architectures and shared-memory paradigms. However, maintaining cache coherence across multiple cores introduces significant microarchitectural complexity. This paper details the design, implementation, and testing of a dual-core, cache-coherent multiprocessor implemented in Verilog. Built around the auto-generated WARP-V CPU core, the architecture features custom first-level data caches (L1D), a shared second-level (L2) cache, and a bus arbiter to manage transactions. Coherence is maintained via a hardware MSI snooping protocol. The system is validated using a suite of custom assembly microbenchmarks designed to stress true sharing, false sharing, and capacity evictions. This project bridges the gap between high-level architectural theory and low-level hardware implementation by designing a fully coherent, dual-core RISC-V processor in Verilog.

1 INTRODUCTION

The widespread adoption of multicore processors within the computing industry has been driven by multiple factors. First, hardware designers encountered strict power constraints that limited their ability to further increase processor frequency without permanently damaging the physical hardware. Second, Moore’s Law (along with rapidly advancing fabrication techniques) has allowed for year-on-year increases in transistor count. Taken together, these two factors led to the industry pivoting to chip multiprocessors in the early 2000s, with companies like Intel and AMD starting to offer consumer products with multiple processing cores on a single die.

The challenge with any system involving multiple independent processing cores, whether it be a chip multiprocessor or one with many separate nodes, is ensuring that all cores have a consistent view of memory while simultaneously utilizing caching to improve processor performance. However, connecting multiple cores together and exploiting thread-level parallelism relies on the fundamental assumption that the memory system will behave in a predictable, predefined manner. This requirement is known as memory consistency, and is concerned with the global ordering of load and store instructions. Consistency, being an architectural specification, is visible to the programmer.

However, *Microarchitects* may implement various structural features to guarantee this consistency. To that end, cache coherence protocols are systems that support consistency models but are invisible to the programmer. Various coherence protocols – or no coherence at all – may be used to uphold a consistency model.

In this paper, we present the design and implementation of a dual-core, cache-coherent multiprocessor in Verilog. In Section 2, we provide background on cache coherence protocols and the RISC-V ISA. In Section 3, we discuss the academic and technical merits of embarking on such a project. Then, in Section 4, we introduce the coherence protocol used in our design, and in Section 5, we examine the specific hardware decisions made to implement the protocol. Finally, in Sections 6 and 7, we discuss the verification and testing of the system, and in Section 8, we summarize our work and discuss future optimizations.

2 BACKGROUND

In each subsequent computer architecture course, students are introduced to more complex and high-performance processor designs. In introductory architectures – ECE250 – we learn about the basic components of a processor and build a simple single-cycle core in Logisim. In digital design – ECE350 – we expand our understanding of hardware design and implement a five-stage, pipelined processor in Verilog. In advanced architectures – ECE552 – we study techniques for performance improvement, such as out-of-order execution, branch prediction, and superscalar execution, and use simulators like GEM5 to evaluate the performance of these techniques. Finally, in advanced architectures II – ECE652 – we cover a wide range of shared-memory multiprocessor designs and parallel architectures.

Of particular interest within the topics covered in ECE652 is cache coherence. As we will discuss in Section 4, cache coherence protocols are systems that maintain a consistent view of memory across multiple cores while still allowing for caching to improve performance. Coherence protocols are invisible to the programmer, but are critical for ensuring that the system behaves in a predictable manner.

However, the complexity of coherence protocols can be difficult to grasp without hands-on experience. By building a cache-coherent multiprocessor in a hardware description language like Verilog, we can gain a deeper understanding of how coherence protocols work and the challenges involved in implementing them.

3 MOTIVATION

The allure of designing hardware has been ever-present in my academic career. In ECE350, we learned how to use Verilog to realize our own circuit designs, and demonstrated that understanding by building a pipelined processor. However, the processor we built had a single-core design, a primitive memory system, and executed a very limited subset of the MIPS instruction set architecture (ISA). Therefore, a natural extension of that project would be to build a more complex multi-core design that supports a more modern and widely-used ISA and a shared memory system.

To that end, RISC-V was chosen as the target ISA for this project.¹ RISC-V is an open-source ISA that has gained significant traction in industry and academia for its ease of use, strong ecosystem of developers and tools, and similarities to ISAs used in education (such as MIPS). Due to RISC-V's similarities to MIPS, the learning curve for implementing a RISC-V core is not steep, as there are many resources available for understanding the ISA and how to implement it. Choosing RISC-V also allows us to leverage a vast array of open-source tools like core generators, assemblers, and compilers, which can significantly speed up the development process.

Experience with Verilog is also a motivating factor for this project. Although our project in ECE350 was undertaken in Verilog, one semester of formal training is insufficient for understanding the nuances and capabilities of a hardware description language. By building a more complex design, we can gain a deeper understanding of how to use Verilog effectively and how to design and implement hardware systems in general.

Finally, the project is motivated by the desire to close the gap between theory and reality. In class, we study various cache coherence protocols and shared-memory multiprocessor designs, often with the assistance of state transition diagrams and other visual aids, but we do not have the opportunity to implement these concepts in hardware. By developing a coherence protocol in Verilog, we can gain a deeper understanding of how these concepts work in practice and the challenges involved in implementing them.

4 CACHE COHERENCE

4.1 Background on Coherence

Cache coherence protocols are systems that enforce two invariants at the cache-block granularity: 1) any given cache block can have at most one writer or multiple readers at any given time, and 2) a read to a cache block must return the most recent value of the data written to the block. These invariants are critical for ensuring that the system behaves in a predictable manner and that all cores have a consistent

view of cache block values. Within cache coherence, two popular solutions exist: snooping protocols and directory protocols.

Snooping protocols require cores to broadcast coherence messages regarding the state of a certain cache block to a shared, totally-ordered bus. Each cache monitors the bus and, upon receiving a relevant coherence message from another core, performs the required action on its own cache. Such protocols are conceptually simple since they abstract away the complexities and challenges associated with a “shared bus” but are difficult to scale.

Directory protocols remove some of the overhead of snooping protocols by adding a data structure known as the directory to the last-level cache. For each cache block, the directory tracks which cores have the block, the state of each block in each core, and the owner of the block. When transactions are issued, instead of the message being put onto a shared bus, it is sent directly to the directory, at which point the transaction becomes ordered and is serviced.

The states tracked by a coherence protocol can be as few as two and up to a very large number (if counting transient states). The simplest protocol (applied generally to a snooping or directory protocol) is a valid/invalid protocol, which tracks whether a block is valid or invalid using a validity bit. While such a system is easy to implement, it leads to excessive overhead from coherence transactions. More effective protocols are those that are a subset of the “MOESI” states. In these protocols, each block is said to be in one of five (or fewer) stable states representing the ownership and sharing information of that cache block. When cores read or write data, the protocol dictates state changes.

4.2 MSI Protocol

The MSI protocol is a subset of the MOESI states and is a common choice for a simple yet effective coherence protocol. It is slightly more complex than a valid/invalid protocol, but significantly improves performance by reducing the number of coherence transactions required to maintain coherence. In the MSI protocol, each cache block can be in one of three states: Modified (M), Shared (S), or Invalid (I). A block is in the Modified state if it is dirty (i.e., it has been written to but not yet written back to the last level cache) and is the only valid copy of that block in the system. A block is in the Shared state if it is clean (i.e., it has not been written to since being loaded from the last level cache) and there can be multiple valid copies of that block in the system. Finally, a block is in the Invalid state if it is not valid (i.e., it has not been loaded from the last level cache into an L1D cache or has been invalidated by another core). The MSI protocol in Table 7.5 from the Primer [1] was chosen for this project due to its simplicity, limited number of transient states, and clearly-outlined state transitions based on hardware events.

4.3 Memory Consistency

Memory consistency models are architectural specifications that define the behavior of memory operations in a shared-memory multiprocessor. They specify the order in which memory operations (loads and stores) appear to execute

1. Specifically, RV32I, which is a 32-bit integer-only ISA.

from the perspective of the programmer. Coherence protocols are one way to implement a consistency model, but they are not synonymous. Thus, it is worth touching on the subject briefly here. RISC-V specifies a relaxed consistency model, which allows for certain reordering of memory operations for performance reasons. However, the MSI protocol we implement in this project instead upholds the sequential consistency model, which is a stronger consistency model that requires all memory operations to appear to execute in a total order that is consistent with the program order of each individual core. This result was mainly a product of the fact that the cores used in this project (discussed more in Section 5) are in-order cores without write buffers, preventing this design from adhering to anything weaker than sequential consistency. Regardless, as microarchitects, we are allowed to implement a stronger consistency model than the one specified by the ISA, so long as we do not violate the guarantees of the weaker model. Therefore, our design is still compliant with the RISC-V specification.

5 HARDWARE ARCHITECTURE

Numerous hardware design decisions were made in order to integrate the myriad components of this project. Each design choice and its associated hardware implementation will be discussed in this section. See Figure 1 for a diagram of the overall architecture and the connections between components.

5.1 CPU Cores

While it would have been possible (and a very good exercise in Verilog design and debugging) to construct a bespoke RISC-V core for this project, the decision was made to use an auto-generated core instead, as the focus of the project was on the interaction *between* cores as opposed to the behavior of a single core. The WARP-V core generator by Redwood EDA [2] was chosen for this project due to the online configurator tool provided by the developers, which allows users to easily customize the core to their needs. The WARP-V core and its associated tools are open-source and utilize Transaction-Level Verilog (TL-Verilog). TL-Verilog [3] is a hardware-description language built upon SystemVerilog that allows for easy design of pipelined CPUs without explicitly declaring latches and wires. TL-Verilog is translated to SystemVerilog using Redwood’s SandPiper tool, which can be accessed online through their MakerChip development tool.

The WARP-V core was configured to be a single-cycle, in-order core without a branch predictor that followed the RV32I instruction set. The core was configured to include a hardware multiplier and hardware divider, but not a floating-point unit, out of concerns for chip area and complexity. The core was configured without built-in caches, and interfaces were manually added to access the program counter, instruction memory, and data memory. As such, the core outputs the address from the instruction memory it would like to read from, and the data memory address, data, and write enable signals. External hardware, using these signals, provides the core with its desired instruction and data and performs the necessary writes to memory. An

interface to inspect the contents of the register file in real-time was also added for debugging and testing purposes.

5.2 Cache Hierarchy

Three cache designs are needed for a project of this sort: 1) a first-level instruction cache (L1I), 2) a first-level data cache (L1D), and 3) a shared second-level cache (L2). The L1I and L1D caches are private to each core, while the L2 cache is shared between the two cores. The cache block size is 16 bytes, or four words. This size was chosen since it provided a large enough block size to allow multiple words to fit within a block while also permitting the use of 128-bit wide data paths without needing to pipeline buses. All caches are initialized to hold “0” values in all blocks upon system startup to prevent undefined behavior (e.g., “X” values in GTKWave).

The L1I cache is a simple, direct-mapped cache implemented using a register array in Verilog. The cache is read-only and is preloaded with instructions using the `$readmemh` Verilog instruction. The L1I cache has a capacity of 1024 words of data, or 4 KB in total.

The L1D cache is a direct-mapped cache implemented using a register array in Verilog. The cache has two sets of ports: one for the core to read and write words, and the other for the cache controller to read and write entire cache blocks. Reads are serviced immediately, while writes are latched on the rising edge of the clock. The L1D holds 256 blocks of data, or 4 KB in total.

The L2 cache is a shared, direct-mapped cache implemented using a register array in Verilog. The cache has one set of ports, as only the bus arbiter is connected directly to the L2 cache. The L2 cache returns a data ready signal upon a successful read or write. An important distinction is that the L2 cache services entire cache blocks, as opposed to individual words, and thus uses 128-bit wide read and write ports. The L2 cache holds 1024 blocks of data, or 16 KB in total.

5.3 Bus Arbiter

The bus arbiter is responsible for managing transactions between the L1D caches and the L2 cache. The bus arbiter implements a round-robin scheme, where each core is given equal priority to access the bus. The priority status is given to the other core upon a core’s completion of a transaction. The bus arbiter serves as the central serialization point for the multiprocessor, managing requests for the shared bus between the cores. To guarantee sequential consistency, the arbiter operates as an atomic, non-split-transaction bus. Once a core is granted access, the bus is locked until the entire memory transaction is fully resolved, preventing overlapping requests from causing data races or protocol deadlocks.

The bus arbiter is a finite state machine (FSM) with four states:

- 1) IDLE
- 2) SNOOP_C1
- 3) SNOOP_C0
- 4) ACCESS_L2

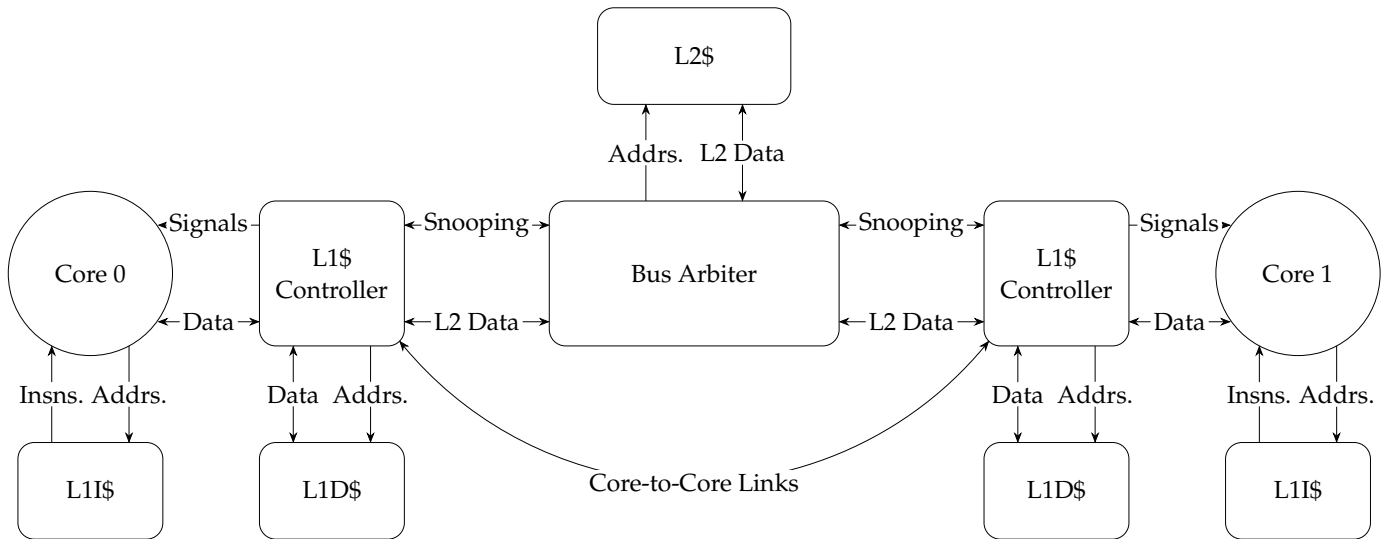


Fig. 1. Cache-coherent multicore architecture

In the IDLE state, the arbiter waits for a transaction request from either core. If both cores request the bus simultaneously, the core priority value dictates which core gets the bus. The arbiter asserts a signal to the core that gets the bus and asserts a snoop request signal to the other core. The FSM then transitions to the corresponding snoop state (e.g., SNOOP_C1 if CORE 0 owns the bus).

In the SNOOP_C0 or SNOOP_C1 states, the arbiter waits for the non-requesting core's snoop response. If the non-requesting core asserts a snoop dirty signal, it indicates that the remote core has a modified copy of the requested block and is sending it directly across the dedicated core-to-core link. In this case, the arbiter immediately sets the bus ready acknowledgment signal and returns to the IDLE state.

In the ACCESS_L2 state, the data must be fetched from or written to the L2 cache. The arbiter transitions to this state and waits for the L2 cache to assert a data ready signal. Once the block is retrieved from the L2 cache, the arbiter sends it to the requesting core, sets the bus ready signal, and transitions back to the IDLE state.

5.4 L1D Cache Controller

The L1D cache controller is responsible for the bulk of the coherence protocol implementation, and is the most intricate component of the design. The cache controller sits between the core and the bus arbiter, alongside the L1D cache, and is responsible for tracking and updating the state of every block in the L1D cache according to the MSI protocol. The finite state machine from Table 7.5 of the Primer [1] is implemented directly in the cache controller module. Each state transition from the table is mapped to a corresponding state transition in the hardware FSM, and the appropriate cache state updates and bus transactions are performed based on the events that trigger the transition. The cache controller also manages the core's interaction with the L1D cache, with the L1D being instantiated within the cache controller module itself and eliminating the need for an additional external interface. Each function of the cache controller is outlined below.

Module Interface

The cache controller has three sets of ports: 1) the core interface, 2) the core-to-core interface, and 3) the arbiter interface. The core interface allows the core (external to this module) to read and write words from the L1D cache. It is comprised of the usual address, data, and write enable signals along with a stall output signal that is used to stall the core while a transaction is being serviced. The core-to-core interface is a 128-bit, bidirectional bus that allows the cache controller to send and receive entire cache blocks to and from the other core in a single cycle in the event of another core having a copy of the requested block. The arbiter interface allows the cache controller to send requests to the bus arbiter and receive data from the bus arbiter. This interface is primarily used to transmit snoop requests and responses between cores and serves as the *only interface* between a core and the shared L2 cache.

Request Register

When a cache miss occurs, the controller must stall the requesting processor while it waits for the bus arbiter. However, as discovered during testing, processor pipelines can be volatile and lead to the simulator hanging. To guarantee transaction stability, we implemented a request latch. The exact cycle at which a miss is detected, the controller locks the CPU's request signals into dedicated internal registers. This isolates the cache controller from the CPU pipeline, providing stable locations for request information for the MSI state machine to evaluate while waiting for the multi-cycle bus response.

Dual Ported L1D Cache

The cache controller must also ensure that its associated core can interact normally with its L1D cache while also satisfying snooping requests from the other core. To support this functionality, the cache controller utilizes the dual-ported L1D cache design described above. A CPU interface handles 32-bit reads and writes, while a separate interface allows the cache controller to work with the bus arbiter to push

128-bit cache blocks between cores and move data between the L1D and L2 caches. By separating these interfaces, the cache controller can service the core's word-level requests while also handling block-level snoop transactions without contention.

Write Merge Optimization

In traditional caches, a store miss requires the controller to fetch the entire block from the L2 cache, write it into the L1D cache, and then perform the store to the block. Implementing this functionality as described proved rather difficult, so an optimization was made to allow the cache controller to perform the store directly to the block as it is being fetched from the L2 cache. When the bus arbiter returns a 128-bit block from the L2 cache, the controller splices the CPU's store data into the correct word of the data based on the address, and then writes the entire block into the L1D cache. This optimization allows store misses to be serviced in a single transaction with the L2 cache.

Miscellaneous

Since bus transactions take multiple cycles to resolve, the cache coherence state machine includes transient states in addition to the three MSI stable states. The controller also tracks capacity misses using an eviction flag. In cases where many addresses map to the same cache index, this flag ensures that blocks that are being evicted are flushed to the L2 cache before being overwritten.

Through testing, the following cycle penalties were observed:

- A cold miss (i.e., a block that has never been loaded into the cache before) incurs a 6-cycle penalty, as the block must be fetched from the L2 cache and written into the L1D cache before the core can proceed.
- A miss to a block that is shared but not modified by another core incurs a 4-cycle penalty, as the block can be fetched directly from the other core without needing to access the L2 cache.
- A hit to a block that is either shared or modified state within a core's local cache incurs a 1-cycle penalty, and the core can proceed with the forwarded data on the next cycle.

Of course, these penalties are all theoretical and rooted in the assumption that the bus arbiter and L2 cache can service requests in a single cycle, which may not necessarily be the case in a real-world deployment. However, these penalties are still useful for understanding the relative costs of different types of misses and hits within the system.

5.5 Top Module

The top Verilog module includes no logic on its own, but rather, ties together the two WARP-V cores, the two cache controllers, the shared L2 cache, the bus arbiter, and the L1I caches. The top module takes in a clock and reset signal and outputs the contents of the register files for both cores for debugging purposes. The top module also includes the instruction memory, which is shared between the two cores and preloaded with instructions using the `$readmemh` Verilog instruction.

6 TESTING AND DEBUGGING

By far the most onerous stage of this project was the testing and debugging phase. The complexity of the design, the number of components, and the intricacies of the coherence protocol made it difficult to identify and fix bugs. To manage this complexity, we use a systematic approach to testing and debugging akin to what we learned in ECE350. First, we verified the functionality of each individual component using simple testbenches written in Verilog. To analyze the output of the system, we used GTKWave to manually inspect signals on wires within the hardware. Myriad issues were encountered upon integrating the components, despite each being tested independently. This result is not necessarily surprising, as the interaction between components can often lead to unforeseen issues. To address this challenge, we wrote specific assembly programs to expose unexpected behavior and then used GTKWave to trace the source of the issue. This process was repeated iteratively until all bugs were resolved and the system behaved as expected. A synopsis of the major bugs encountered is outlined below.

WARP-V Modifications

Due to the need to expose the instruction memory address, data memory address, and data signals from the WARP-V core, modifications were made to the core's Verilog code to read and write directly from and to autogenerated internal signals. The most significant issue of this sort was a multiple-driver error caused by existing logic and new external L1D interfaces both trying to drive the same internal signals. This issue was resolved by removing the existing logic and replacing it with the new external interfaces completely, but finding the source of the issue was difficult due to the sheer size of the autogenerated code ².

Coherence Protocol Edge Cases

Once the cores were functional within the top-level Verilog module, and could be seen interacting with instruction and data memory, the next step was to test the sharing of data. Very quickly, issues emerged. The first issue concerned a core reading a shared location another core had written and stalling indefinitely, breaking the simulator. Tracing the issue back to its source revealed that the controller transitioned into a transient state without asserting a bus request, causing the arbiter to never send snoop requests to the other core and thus never resolving the transaction. This issue was resolved by ensuring that the appropriate bus request was asserted during the state transition.

The most severe coherence bug was data getting lost during consecutive memory writes. If CORE 0 wrote to a shared location and subsequently wrote to a synchronization flag residing in the same 16-byte cache block, the data was destroyed. When the core issued the request to write to the flag, the arbiter correctly invalidated CORE 1's value. However, the L1 controller mistakenly fetched a stale 128-bit block of zeroes from the bus and overwrote the valid data in the L1D cache.

². Despite being a single-cycle, scalar, in-order core with neither bells nor whistles, the autogenerated code is still roughly 4,000 lines of Verilog!

To solve this issue, we separated the bus ready signal into two distinct signals: a true cache fill and a permission upgrade. We modified the L1D cache to ignore the 128-bit write enable unless it was truly a cache fill. This modification allowed the main cache port to write the local 32-bit flag value into the cache without corrupting the surrounding data with undesirable values from the bus.

In general, these bugs were resolved by running many versions of a particular assembly test program to isolate the sequences of instructions or events that caused the issue. GTKWave proved to be an invaluable tool for tracing back the source of the issue, as it allowed us to inspect the values of signals at each cycle and understand how the system was behaving at any given time. The assembly programs themselves, in their original incarnations, were padded generously with `nop` instructions to allow for values to propagate through the system, and were gradually removed as our confidence in the system's stability increased.

7 EVALUATION

Programming for this system was a unique challenge in and of itself. Each benchmark had to be written specifically for the core it was run on, as the cores do not share an instruction memory and thus cannot run the same program. Further, due to the absence of an operating system or autotester (like those seen in earlier ECE classes), programs had to be written from scratch and analyzed upon completion. Each program was written in RISC-V assembly or C and assembled using the GNU assembler. The resulting machine code was then converted to hexadecimal and preloaded into the instruction memories of the cores. The benchmarks are described below.

7.1 Microbenchmarks

Several microbenchmarks were written in RISC-V assembly to verify that the system behaved as expected under various sharing scenarios. The microbenchmarks were designed to test independent operation, simple sharing, false sharing, and capacity evictions.

Independent Operation

This benchmark was designed to verify that the cores could operate independently without sharing any data, and was tested before studying the sharing behavior of the system. Each core simply performed a series of computations on local data and stored the results in its own register file. The benchmark was successful, as both cores were able to execute their instructions and produce the expected results without any interference from the other core. This fundamental test provided confidence that the cores and top-level module were functioning as expected when no sharing occurred, and was a necessary first step before testing the more complex sharing scenarios.

Simple Sharing

This benchmark was designed to test the behavior of the system when two cores share a single cache block. Several basic benchmarks were used, with the simplest test case having CORE 0 write to a shared location and then, some time later,

CORE 1 reading the shared location. This benchmark was successful, as CORE 1 was able to read the value written by CORE 0. This result was verified by a test in which CORE 0 wrote four values to words within the same block, CORE 1 read those values into its register file, and summed them together, producing the expected result. Finally, a more rigorous test was created by placing the data value and the synchronization flag adjacent within the same cache block. The test forced the block to constantly move across the core-to-core link. As the producer generated data and the consumer read it, the benchmark systematically triggered state transitions, including initial write misses from I to M, sharing from M to S, and permission upgrades from S to M without deadlocking. These tests were successful and provided confidence that the system could maintain coherence under simple sharing scenarios.

False Sharing

This benchmark evaluated the cache controllers and bus arbiter under the extreme contention of false sharing. In this scenario, CORE 0 and CORE 1 executed loops consisting of fifty iterations, continuously writing to distinct 32-bit words that map to the same cache block. Because our MSI protocol tracks permissions at the block granularity, these independent writes forced the cache block to repeatedly traverse the core-to-core link. Each store instruction immediately triggered a miss, requiring the executing core to broadcast a request to invalidate the other core so it can write. The successful completion of both core programs verifies that our round-robin arbitration policy prevents protocol deadlocks and starvation even during high contention.

Capacity Evictions

This benchmark evaluated the L1 cache controller's capacity miss detection and dirty writeback mechanisms. In this test, CORE 0 executed a loop writing data values to 257 sequentially mapped cache blocks. Because our L1 data cache has a maximum capacity of 256 blocks, the 257th write forced a conflict with the first modified block. The L1 controller must stall the processor, assert its eviction flag, and flush the dirty 128-bit cache block to the L2 cache. Once the eviction was completed and the 257th write finished, CORE 0 wrote to a flag to unblock CORE 1. CORE 1 then attempted a read to the originally-evicted address. The successful retrieval of the original data by CORE 1 demonstrates that the evicted block was correctly stored in the L2 cache and routed to the requestor, proving that the system can handle capacity misses without data loss or coherence violations.

7.2 Real-World Benchmarks

Several real-world benchmarks were also written in C and compiled to RISC-V assembly to further evaluate the system's behavior under more realistic workloads. These benchmarks included a single-threaded array sum program, a single-threaded factorial program (to test the multiplier), and a multi-threaded matrix multiplication program.

Array Sum

This C benchmark evaluates the processor's ability to handle sequential memory accesses and basic arithmetic loops.

The program allocates and initializes an array of integers, forcing the processor to execute a sequence of initial cache write misses followed by immediate read hits. By computing the sum of these elements and asserting a flag, this benchmark validates that basic C programs can be run on this processor.

Factorial

To verify the functionality of the multiplier unit, which was added as an extension to the WARP-V core, we use a C program that computes $10!$. Unlike memory-intensive applications, this program relies on repeated multiplier use. The successful execution of this benchmark demonstrates that the multi-cycle multiplication execution units are functioning correctly.

Matrix Multiplication

Finally, we evaluated the system under a realistic, parallelized workload using a shared-memory matrix multiplication benchmark. In this test, a 4×4 matrix operation is divided across both cores. CORE 0 initializes the matrices in the shared L2 cache and computes the top half of the result, while CORE 1 waits on a flag before computing the bottom half. This benchmark acts as a stress test for the entire multi-core processor. It forces true sharing transitions, as CORE 1's reads trigger snooping requests that downgrade CORE 0's cache lines from the modified to the shared state, activating the core-to-core data link. The successful verification of both cores' independent results proves that our MSI protocol, transient state logic, and bus arbiter operate correctly under parallel workloads, and that the system can run realistic multithreaded programs without coherence violations.

7.3 Benchmarking Toolchain and Challenges

Executing C code on this multicore processor requires a specialized software toolchain, as the system lacks an operating system to manage memory layout or thread initialization. First, core-specific assembly boot files were required to manually initialize independent stack pointers before transferring control to the C routines, preventing the cores from corrupting each other's local variables in the shared L2 cache. Second, a linker script was necessary to map the compiler's generated code to our physical hardware addresses, explicitly routing the instruction memory to the independent L1 instruction caches and the variable memory to the shared L2 data cache. Finally, shell scripts compiled the programs and extracted the .hex files, and compiled the entire Verilog simulation.

While our custom microbenchmarks successfully validate the functional correctness of the MSI coherence protocol, rigorously evaluating the performance of the overall architecture is rather difficult. The main challenge lies in comparing the performance of our system to other systems, as 1) designing another system by hand in Verilog would be challenging, and 2) the architecture of this system is unique and would not be worthwhile to replicate at a larger scale without significant modifications. Because the processor operates on bare metal without an operating system, porting industry-standard multithreaded benchmark

suites is currently impractical. Also, comparing our Verilog implementation to high-level architectural simulators is difficult since it would require translating the behavior of the Verilog implementation into a software model. To enable a deeper analysis of the memory system, future iterations of this project must integrate dedicated hardware performance monitoring counters so that detailed profiling can be conducted. Specifically, implementing counters to track coherence-related events – such as the total number of bus requests, invalidations, capacity evictions, and core-to-core data transfers – would provide valuable empirical data. These built-in monitors would allow us to quantify the overhead of the MSI protocol and identify system behavior under demanding workloads.

8 CONCLUSION AND FUTURE WORK

The design and implementation of a dual-core, cache-coherent multiprocessor in Verilog was presented in this paper. The system was designed to uphold the MSI coherence protocol and was tested using a variety of microbenchmarks to verify its correctness under various sharing scenarios. As such, the project can run programs compiled to RISC-V assembly that share data between the cores without coherence violations. However, there are several areas for future work and optimization.

Comprehensive Benchmarking

While the microbenchmarks described in Section 7 were successful in verifying the correctness of the system under various sharing scenarios, they were not comprehensive. Future work could involve developing a more extensive suite of benchmarks (such as SPLASH-2) that test a wider range of sharing scenarios. This would provide a more thorough evaluation of the system's performance and shortcomings under more realistic workloads. Unfortunately, many traditional multithreaded programs use operations like `clone`, through which one process creates a child process that shares the same memory space. Since our system does not support such operations, we would need to write individual programs for each core that use shared memory for communication and synchronization, which would be a non-trivial task.

Register File Inspector and Autotester

Currently, the only detailed way to verify the correctness of a program running on the system is to inspect the register file outputs in GTKWave and compare them to the expected values. This process is time-consuming and subject to human error, as it requires manually analyzing the waveforms and keeping track of the expected values. It would be advantageous to develop an autotester (in Python, for example) that automatically interfaces with the register file outputs provided in the top module and compares against expected values. This change would streamline the testing process and reduce the likelihood of human error when inspecting GTKWave traces.

Synthesis and Physical Implementation

The original goal of this project was to synthesize the design and implement it on an FPGA. However, due to time constraints and the realization that running this processor as built on an FPGA would not permit functionality beyond that already implemented in simulation, this step was not completed. We began the synthesis process in Vivado; however, the use of `$readmemh` instructions caused a lot of issues, so in the interest of time, this effort was tabled in favor of more benchmarking in Icarus Verilog. The jump from simulation in Icarus Verilog to synthesis in AMD Vivado would be feasible and may be undertaken in the near future. Future work could involve developing an interface that would allow the dual-core processor running on the FPGA to interact with external devices, much like in ECE350. Importantly, implementing the design in hardware would necessitate some way of inspecting the state of the system, either using internal counters, debug cores, or outputting signals on wires that could be read by an oscilloscope. Physical implementation would provide insights into the challenges of designing and implementing hardware systems, such as timing constraints, power consumption, and area optimization, which are not fully captured in simulation.

Directory Coherence Protocol

Although snooping protocols are conceptually simpler and perform well for small-scale systems, they do not scale well as the number of cores increases. In the future, implementing a directory-based coherence protocol could sharpen our understanding of directory coherence protocols in practice as opposed to only in theory. This change would require significant modifications to the cache controllers and bus arbiter, as well as the creation of a directory data structure to track the state of each cache block across all cores. However, it would provide valuable insights into the design and implementation of more complex coherence protocols and their associated hardware components.

Parameterization and Configurability

In conjunction with creating a directory-based coherence protocol, parameterizing the design would allow for easy configuration of various aspects of the system, such as the number of cores, cache sizes, and block sizes. This change would make the design more flexible and adaptable to different use cases and workloads, and would also provide insights into the trade-offs involved in designing hardware systems with varying parameters.

High-Performance Cores

The WARP-V cores used in this project are simple, single-cycle, in-order cores without any performance optimizations. A natural next step would be developing more complex cores with features such as pipelining, out-of-order execution, and wide execution paths, and branch predictors. This change would require significant modifications to the core design and the cache controllers to handle the increased complexity of the cores, but would provide valuable insights into the design and implementation of

high-performance processors and their associated hardware components. Though not necessarily the goal of this project, striving for more performance is generally a worthwhile endeavor in computer architecture and would give us a deeper understanding of the challenges and trade-offs involved in designing high-performance hardware systems.

Improved Programmability

As built, the system is currently not very programmable, as it requires writing assembly programs from scratch for each core. Designing a more user-friendly programming interface, such as a simple operating system or runtime environment that allows for easier development and execution of multithreaded programs on the system, would open up new possibilities for software development and deployment. This change would require significant modifications to the software stack and the hardware design to support the necessary features, but would make it far easier to port existing multithreaded programs and industry-standard benchmarks to the system, allowing for more rigorous testing and evaluation of the system's performance and behavior under realistic workloads.

REFERENCES

- [1] Daniel J. Sorin, Mark D. Hill, and David A. Wood, "A Primer on Memory Consistency and Cache Coherence," Synthesis Lectures on Computer Architecture, Morgan & Claypool Publishers, 2nd ed., 2016.
- [2] Hoover, Steve. "WARP-V." *GitHub Repository*. <https://github.com/stevehoover/warp-v>
- [3] "TL-X." *Transaction-Level Design*. <https://www.tl-x.org>